

Atividade 1

Instruções gerais

Antes de começar, revejam a **página 23 do livro *flex e bison*** (Levine, O'Reilly), que traz o exemplo em que esses programas se baseiam.

Todas as questões pedem a implementação de um especificador `.l` completo (seções de definições, regras e código do usuário), compilável com o fluxo usual:

```
flex programa.l
gcc lex.yy.c -o programa
./programa (Ctrl+D para finalizar a entrada)
```

Entrega

Todo o código desta atividade vai para um **repositório no GitHub**, dentro de uma pasta `AT1` com uma subpasta por questão:

```
AT1/
  q1/
    q1.1      especificador flex da questao
  q2/
    ...
  q3/
  q4/
```

Cada pasta de questão precisa conter:

- (1) O arquivo `.l` resolvendo a questão. Quando os itens (a), (b) e (c) se acumulam sobre o mesmo programa, entreguem um único `.l` com tudo já integrado.

Não adicionem arquivos gerados: `lex.yy.c` e os executáveis ficam de fora, use um `.gitignore`. O que é entregue é o `.l`, que é a fonte, mais as entradas e saídas de teste.

O repositório precisa estar **público** no momento da correção. Enviem o link como resposta da atividade no Classroom, conforme combinado em aula.

Questões

Questão 1. Escrevam um contador de estatísticas de texto, no estilo do comando `wc` do Unix. O programa lê a entrada padrão até o fim e, quando a entrada termina, imprime um relatório com as contagens acumuladas. Comecem pelo caso básico, com três contadores globais:

- **linhas:** uma unidade a cada `\n` lido.
- **palavras:** uma unidade a cada sequência maximal de letras (use regex). A sequência `ola mundo` conta duas palavras, por exemplo.

- **caracteres:** todos os caracteres lidos, incluindo espaços, pontuação e quebras de linha. Use `yyvaleng` para somar o tamanho do lexema em vez de somar de um em um.

Sobre esse programa base, acrescentem as contagens abaixo:

- (a) Conte números inteiros (ex.: 42) e números de ponto flutuante (ex.: 3.14) em contadores distintos.
- (b) Conte separadamente os seguintes sinais de pontuação (. , ; : ! ?), que de outro modo cairiam na regra genérica que casa qualquer caractere avulso. Atenção: o ponto de 3.14 pertence ao número e não deve entrar nessa contagem, o que depende da ordem em que vocês escrevem as regras.

O relatório final é único, impresso apenas quando a entrada acaba, com um valor por linha. Para a entrada:

```
0 valor e 3.14 e o total, 42 itens!
Fim da contagem.
```

a saída esperada é algo no estilo:

```
linhas: 2
palavras: 10
caracteres: 53
inteiros: 1
floats: 1
pontuacoes: 3
```

Questão 2. Implemente um analisador léxico para os tokens de uma calculadora simples. O programa deve ler expressões da entrada padrão e, para cada token reconhecido, imprimir uma linha indicando seu tipo: **PLUS** para +, **MINUS** para -, **TIMES** para *, **DIVIDE** para /, **ABS** para o caractere de valor absoluto |, **NUMBER** seguido do lexema para sequências de dígitos, e **NEWLINE** ao final de cada linha. Espaços e tabulações devem ser ignorados, e qualquer caractere não coberto pelas regras acima deve gerar a linha **Mystery character** seguida do caractere. Por exemplo, para a entrada `3 + 4 * 2`, a saída esperada é algo no estilo:

```
NUMBER 3
PLUS
NUMBER 4
TIMES
NUMBER 2
NEWLINE
```

A partir desse programa base, amplie o conjunto de tokens reconhecidos, mantendo o mesmo padrão de saída (um nome de token por linha):

- (a) Adicione o reconhecimento de parênteses, imprimindo **LPAREN** para (e **RPAREN** para).
- (b) Adicione os operadores % (módulo) e ^ (potência), imprimindo **MOD** e **POWER**, respectivamente.

- (c) Adicione os operadores relacionais `<`, `>`, `<=`, `>=`, `==` e `!=`, imprimindo `REL_OP` seguido do operador reconhecido (ex.: `REL_OP <=`). Preste atenção à ordem das regras para que `<=` não seja quebrado em dois tokens.

Com os três itens implementados, a entrada:

```
(3 + 4) % 5 ^ 2
7 <= 10 @
```

deve produzir:

```
LPAREN
NUMBER 3
PLUS
NUMBER 4
RPAREN
MOD
NUMBER 5
POWER
NUMBER 2
NEWLINE
NUMBER 7
REL_OP <=
NUMBER 10
Mystery character @
NEWLINE
```

Observe que no fim da segunda linha: o `@` não pertence à linguagem da calculadora, então cai na regra genérica e vira `Mystery character @`, sem interromper a análise. O `NEWLINE` vem depois dele, porque a quebra de linha só é lida em seguida.

Questão 3. Implemente um novo analisador léxico para uma mini-linguagem de programação, no mesmo formato de saída da questão anterior (um token por linha). O programa precisa reconhecer:

- (a) As palavras reservadas `if`, `else`, `while` e `return`, imprimindo o nome do token em maiúsculas (ex.: `IF`).
- (b) Identificadores, ou seja, uma sequência iniciada por letra ou `_` e seguida de letras, dígitos ou `_`, que *não* seja palavra reservada, imprimindo `ID <lexema>`.
- (c) Números, inteiros ou com ponto decimal, imprimindo `NUMBER <lexema>`; e comentários de linha iniciados por `//`, que vão até o fim da linha e devem ser descartados.

Espaços, tabulações e quebras de linha são ignorados. Qualquer outro caractere deve gerar a linha `SIMBOLO` seguida do caractere. Para a entrada:

```
// media dos valores
x = 3.5;
if (x > 1) return x;
```

a saída esperada é algo no estilo:

```
ID x
SIMBOLO =
NUMBER 3.5
SIMBOLO ;
IF
SIMBOLO (
ID x
SIMBOLO >
NUMBER 1
SIMBOLO )
RETURN
ID x
SIMBOLO ;
```

A linha de comentário não gerou saída nenhuma. Reparem que `x` virou `ID`, enquanto `if` e `return` viraram tokens próprios, e que a ordem das regras é o que impede `if` de ser reconhecido como identificador.

Questão 4. Modifique o analisador da **Questão 3** para que ele funcione sobre arquivos em vez de sobre a entrada padrão.

- (a) O programa deve receber o nome do arquivo de entrada como argumento de linha de comando (`argv[1]`), abri-lo com `fopen` e redirecionar `yyin` para esse arquivo antes de chamar `yylex()`.
- (b) Se nenhum argumento for passado, ou se o arquivo indicado não existir ou não puder ser aberto, o programa deve imprimir uma mensagem de erro clara na saída padrão e encerrar com código de saída diferente de zero, sem chamar `yylex()`.

Antes de enviar, confirmam: cada questão tem seu `.1`, sua entrada e sua saída; cada programa compila partindo do zero, com `flex` e depois `gcc`, em uma cópia limpa do repositório; `lex.yy.c` e os executáveis não foram versionados; o repositório está público e o link enviado no Classroom abre em uma janela anônima do navegador.